

MATHEMATICAL MODELLING FOR MACHINE LEARNING PERSONAL PROJECT: BINARY CLASSIFICATION

In this notebook I will construct a model which aims to predict a binary label (0,1) for a dataset with 25 generic numerical features and 2 generic categorical features. My task will be to compare the performance of a linear Support Vector Machine (for example `sklearn.svm.LinearSVC` or `sklearn.svm.SVC` with a linear kernel) with that of Logistic Regression (`sklearn.linear_model.LogisticRegression`). The dataset is a dataframe of 1400 rows and 27 columns. My notebook will follow this structure:

- EDA (Exploratory Data Analysis)
- Data Preprocessing
- Model Selection
 - Extra Models
- Hyperparameter Tuning
- Model Fitting
- Shap model visualization
- Final Remarks and Limitations

I will include at the start of each section a markdown written by ChatGPT saying what it would have done in my place and then briefly commenting it.

```
In [1]: import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

from sklearn.linear_model import LogisticRegression
from scipy.stats import loguniform
from sklearn.model_selection import train_test_split, cross_val_score, Ra
from sklearn.preprocessing import StandardScaler, OneHotEncoder, Polynomi
from sklearn.metrics import ConfusionMatrixDisplay, balanced_accuracy_sco
from sklearn.ensemble import RandomForestClassifier
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.feature_selection import mutual_info_classif
from sklearn.svm import LinearSVC, SVC
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE
from sklearn.impute import SimpleImputer

random_state = 44
```

```
In [2]: train_data = pd.read_csv('mldata_0003314701.csv')
```

EXPLORATORY DATA ANALYSIS

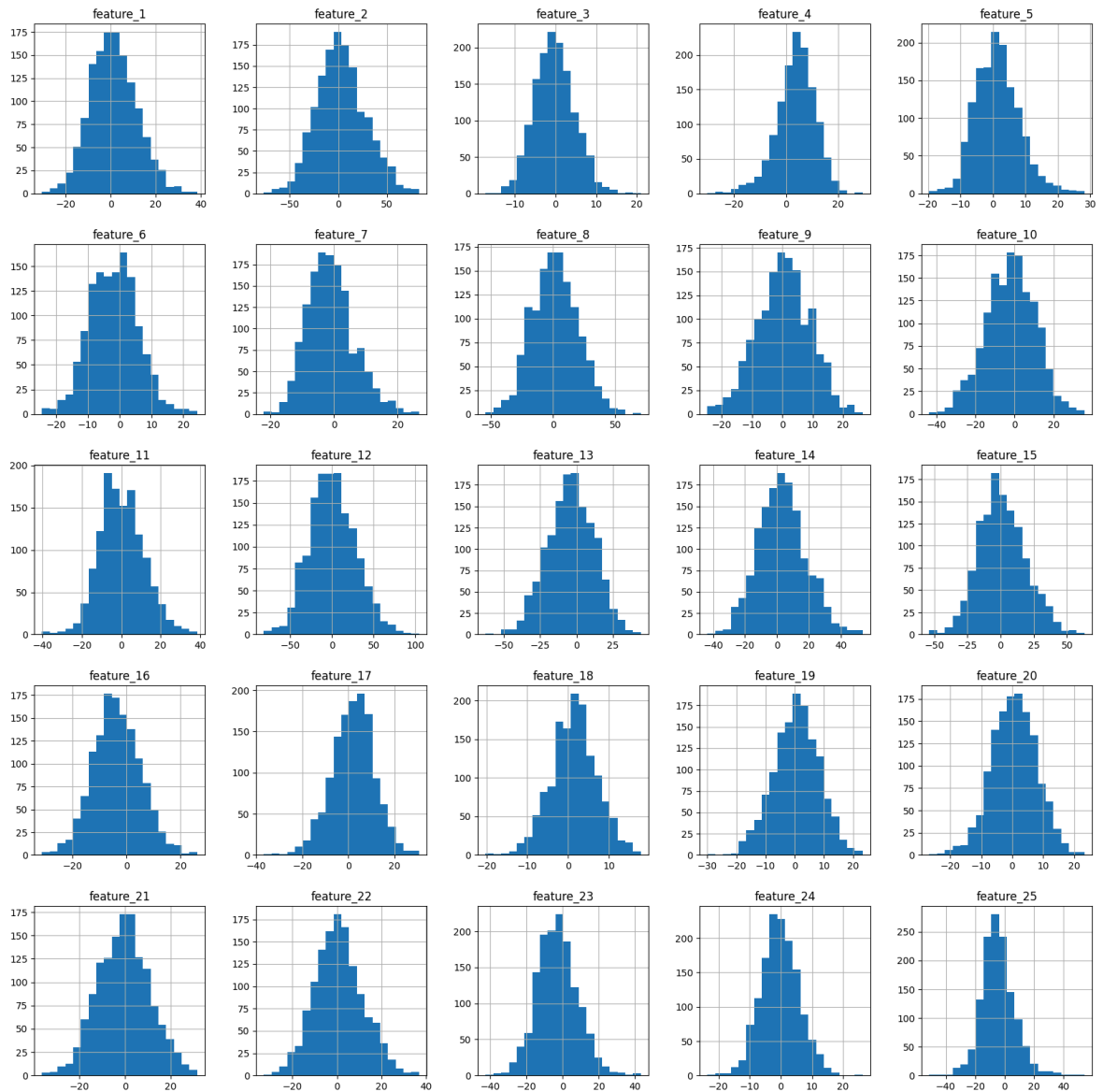
Firstly, I separate the training data into `x_train` and `y_train`, with `x` being the features and `y` the target (namely the labels). I also create the variables `num_cols` and `cat_cols` to separate the numerical features columns and the categorical features ones. In this section i will include some plots which are going to be useful for my preprocessing pipeline:

```
In [3]: x_train = train_data.drop(['label', 'id'], axis=1)
y_train = train_data['label'].squeeze()
cat_cols = x_train.select_dtypes(include=['object']).columns
num_cols = x_train.select_dtypes(include=['int64', 'float64']).columns
```

```
In [4]: display(x_train.describe())
x_train.hist(figsize=(20, 20), column=num_cols, bins=20)
plt.show()
```

	feature_1	feature_2	feature_3	feature_4	feature_5	feature_6
count	1324.000000	1400.000000	1400.000000	1290.000000	1400.000000	1272.000000
mean	1.280572	3.061295	-0.259094	3.662490	1.403695	-1.823094
std	10.318960	24.949568	4.911310	7.513987	6.804337	7.508094
min	-30.508517	-76.203409	-17.189057	-30.210852	-19.592124	-24.373094
25%	-5.904726	-13.797621	-3.604747	-0.589990	-3.441849	-7.268094
50%	0.818062	1.973863	-0.455461	4.274168	1.125511	-1.755094
75%	8.097738	18.796333	2.800158	8.650247	5.538092	3.189094
max	38.369724	82.529503	20.923396	29.342364	28.169022	24.343094

8 rows x 25 columns

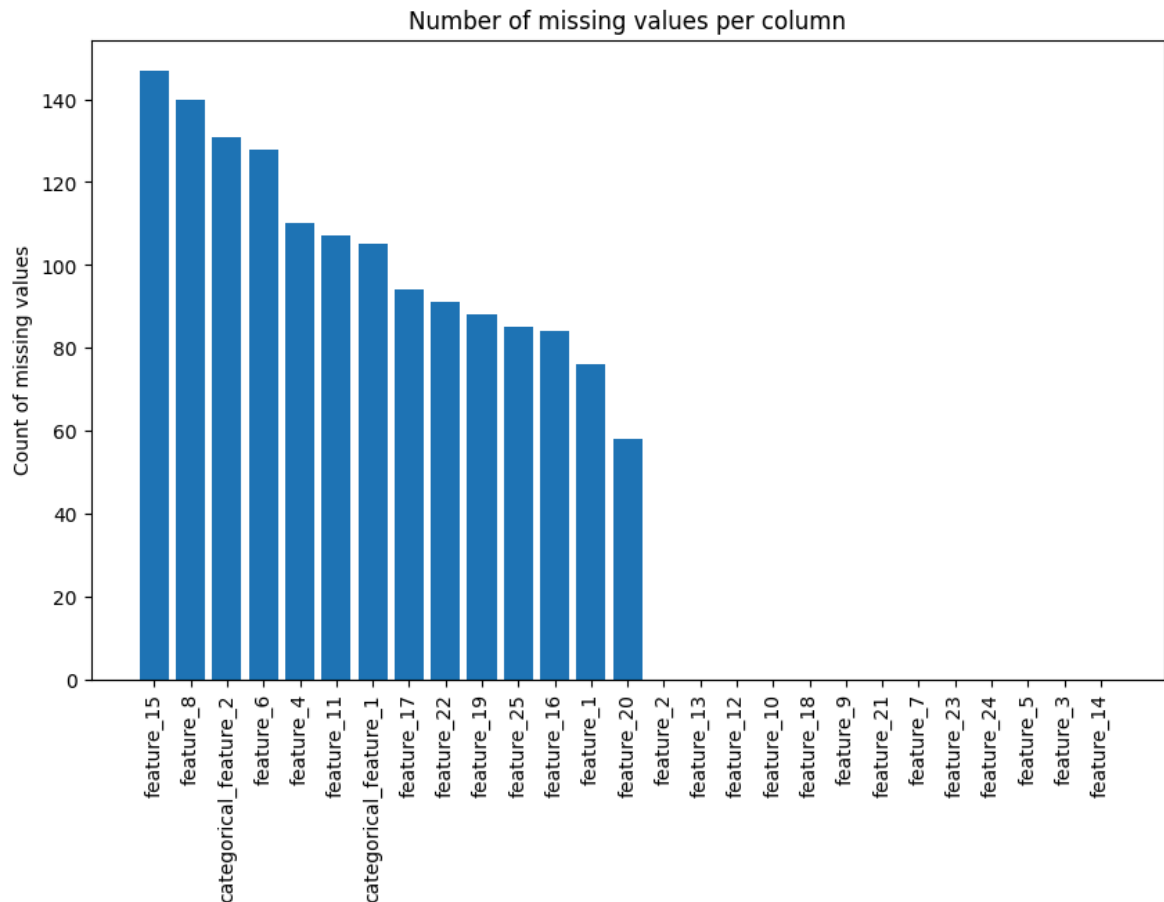


We see that all the numerical features resemble a normal distribution and it is not possible to spot clear and visible outlier for outlier detection.

MISSING VALUES

I now plot the missing values in descending order.

```
In [5]: missing_values = x_train.isnull().sum().sort_values(ascending=False)
plt.figure(figsize=(10, 6))
plt.bar(missing_values.index, missing_values.values)
plt.xticks(rotation=90)
plt.title('Number of missing values per column')
plt.ylabel('Count of missing values')
plt.show()
```



```
In [6]: missing_labels = y_train.isnull().sum()
print(f"Number of missing values in the target variable: {missing_labels}")
```

Number of missing values in the target variable: 0

The number of missing values per feature is at most about 140 for x , which is approximately 10% of the 1,400 observations. This number is not extreme, so imputation is a reasonable approach rather than dropping rows or columns.

MUTUAL INFORMATION

For a discrete random variable X with support $\mathcal{X}$, and pmf $p(X)$, we define entropy and we denote by $H(X)$ the quantity:

$$H(X) = \mathbb{E}[I(p(X))] = \sum_{x \in \mathcal{X}} p(x) \log_2 \left(\frac{1}{p(x)} \right)$$

for $I(p(X))$ equal to:

$$I(p(x)) = \log_2 \left(\frac{1}{p(x)} \right)$$

and denoted as Information Content. We then define Conditional Entropy for two random variables X and Y :

$$H(X | Y) = \sum_{x \in \mathcal{X}, y \in \mathcal{Y}} p(x, y) \log_2 \left(\frac{1}{p(x | y)} \right) = \sum_{y \in \mathcal{Y}} H(X | Y = y) p(y).$$

and the joint entropy as:

$$H(X, Y) = \sum_{x \in \mathcal{X}, y \in \mathcal{Y}} p(x, y) \log_2 \left(\frac{1}{p(x, y)} \right)$$

Finally, the mutual information between X and Y is defined as:

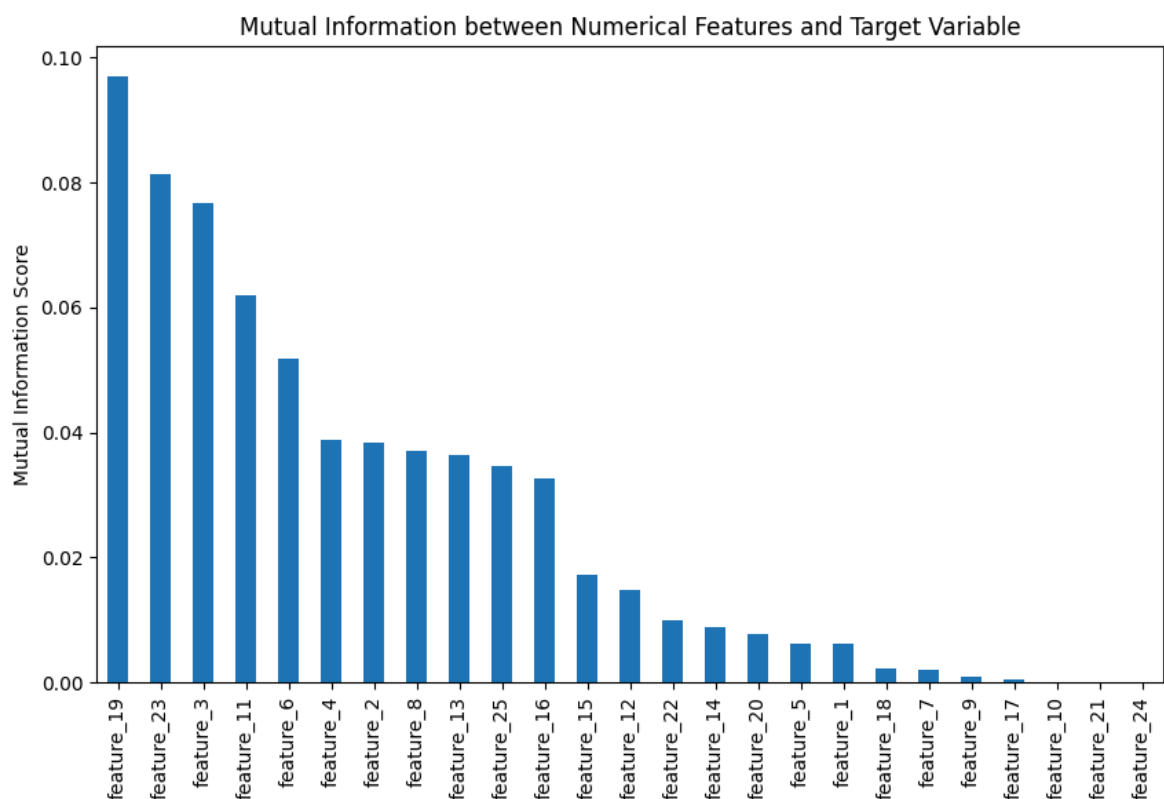
$$I(X; Y) = H(X) - H(X | Y)$$

Equivalently,

$$I(X; Y) = H(X) + H(Y) - H(X, Y)$$

Mutual information measures how much knowing one variable reduces uncertainty about the other. In this project, I use it as an exploratory measure of how informative each feature is about the label.

```
In [7]: mutual_info = pd.Series(mutual_info_classif(x_train[num_cols].fillna(x_train[num_cols]), y_train, 20, 10, 1000000))
mutual_info.sort_values(ascending=False, inplace=True)
plt.figure(figsize=(10, 6))
mutual_info.plot(kind='bar')
plt.title('Mutual Information between Numerical Features and Target Variable')
plt.ylabel('Mutual Information Score')
plt.show()
```



CORRELATION

Next step is to include the correlation between the numerical columns

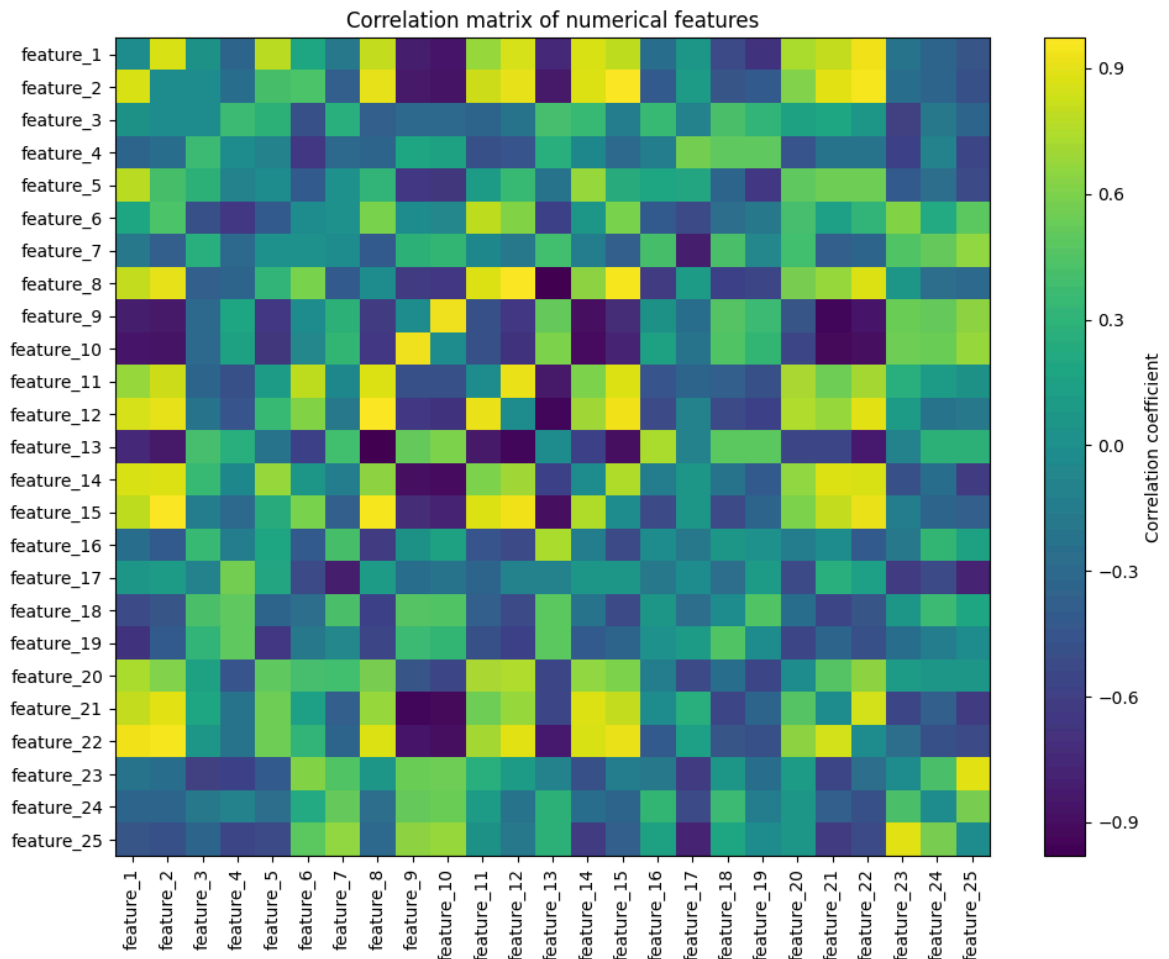
```
In [8]: corr_matrix = x_train[num_cols].corr()

for i in range(len(corr_matrix.columns)):
```

```

corr_matrix.iloc[i, i] = 0 # we remove the diagonal because it is n
plt.figure(figsize=(10, 8))
plt.imshow(corr_matrix, aspect='auto')
plt.xticks(range(len(num_cols)), num_cols, rotation=90)
plt.yticks(range(len(num_cols)), num_cols)
plt.title('Correlation matrix of numerical features')
plt.colorbar(ticks=[-0.9, -0.6, -0.3, 0, 0.3, 0.6, 0.9], label='Correlati
plt.tight_layout()
plt.show()

```



```
In [9]: corr_matrix.unstack().sort_values(ascending=False).drop_duplicates()
```

```

Out[9]: feature_15 feature_2      0.972095
feature_8  feature_12     0.966370
feature_22 feature_2      0.942970
feature_8  feature_15     0.941818
feature_10 feature_9      0.940595
...
          feature_14    -0.914130
          feature_21    -0.923836
feature_12 feature_13    -0.940905
feature_21 feature_9     -0.947006
feature_8  feature_13    -0.979975
Length: 301, dtype: float64

```

I notice that i have several pairs of features that are highly correlated with each other: i now plot the scatter plot for the 5 most correlated (correlation coefficient close to 1) and the 5 most inversely correlated (correlation coefficient close to -1).

```
In [10]: corr_pairs = corr_matrix.unstack().sort_values(ascending=False)
corr_pairs = corr_pairs[corr_pairs != 0]
corr_pairs = corr_pairs[~corr_pairs.index.map(lambda x: tuple(sorted(x)))

top_positive = corr_pairs.head(5)
top_negative = corr_pairs.tail(5)

top_positive, top_negative
```

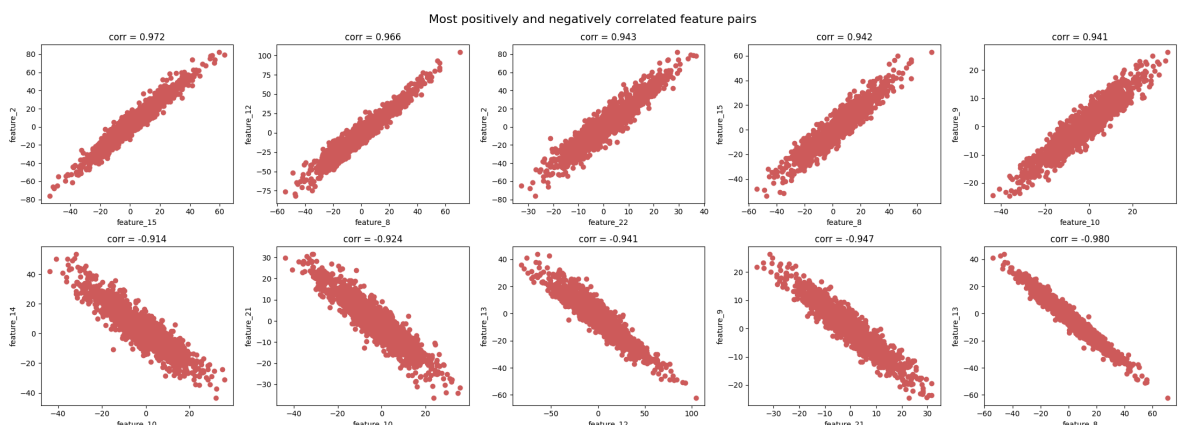
```
Out[10]: (feature_15 feature_2      0.972095
feature_8 feature_12      0.966370
feature_22 feature_2      0.942970
feature_8 feature_15      0.941818
feature_10 feature_9      0.940595
dtype: float64,
feature_10 feature_14     -0.914130
feature_21 feature_21     -0.923836
feature_12 feature_13     -0.940905
feature_21 feature_9      -0.947006
feature_8 feature_13      -0.979975
dtype: float64)
```

```
In [11]: pairs_to_plot = pd.concat([top_positive, top_negative])

fig, axes = plt.subplots(2, 5, figsize=(22, 8))
axes = axes.ravel()

for ax, ((feature_x, feature_y), corr_value) in zip(axes, pairs_to_plot.i
ax.scatter(x_train[feature_x], x_train[feature_y], alpha=1, color = 'r'
ax.set_xlabel(feature_x)
ax.set_ylabel(feature_y)
ax.set_title(f"corr = {corr_value:.3f}")

plt.suptitle("Most positively and negatively correlated feature pairs", f
plt.tight_layout()
plt.show()
```



These feature pairs show very strong positive or negative correlation. Hence it is reasonable to try PCA as a dimensionality-reduction experiment, for data visualization and an overall simpler model representation.

LLM VERSION

The purpose of exploratory data analysis is to understand the dataset before modeling. I would start by plotting the distribution of the target variable. This gives an immediate view of whether one class dominates the other.

Next, I would examine missing values. Missingness can occur in both numerical and categorical variables, and the amount of missingness can influence the choice of preprocessing. For numerical features, median imputation is usually a strong default because it is robust to outliers. For categorical features, replacing missing values with the most frequent category is a simple and effective baseline.

I would then inspect the numerical features. This can include summary statistics, histograms, boxplots, and correlation analysis. Histograms help reveal skewed distributions, while boxplots can highlight outliers. A correlation heatmap can show whether some numerical features are strongly related to one another. Highly correlated variables are not necessarily a problem for tree-based models, but they can affect linear models and can also help explain model behavior.

For categorical features, I would check the number of unique categories and compare the category levels between the training set and the hidden test set. This is important because the hidden test set may contain categories that were not seen during training. To handle this safely, the one-hot encoder should be configured with `handle_unknown="ignore"` so that unseen categories do not cause prediction errors.

I would also examine the relationship between features and the target. For numerical variables, boxplots grouped by target class can show whether a feature separates the two classes. For categorical variables, target-rate plots can show whether some categories are associated with a higher probability of class 1.

The EDA stage should not be used to make final decisions based on the hidden test labels, because those labels are not available. It should mainly guide preprocessing choices, feature understanding, and sanity checks.

COMMENTS

We agreed on the choices to make for EDA; it's not surprising as with a synthetic dataset my work is limited by my not knowledge of what the data signifies. Moreover ChatGPT 5.5 didn't check for Mutual Information or expanded the correlation analysis.

PREPROCESSING

As we don't have any information on what the dataset represents, I opt for a standard preprocessing pipeline: filling NAs with the most frequent value for categorical columns and with the median for numerical columns. I chose these metric because they are the industry standard; in particular the median is a very robust solution handling well sets with outliers (I haven't spotted any but I still feel like it's the safer

option). I then created Dummy Variables columns using `OneHotEncoder` for categorical variables and standardized the numerical ones.

```
In [12]: cat_preprocessor = Pipeline([
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('encoder', OneHotEncoder(handle_unknown='ignore'))
])

num_preprocessor = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())
])

preprocessor = ColumnTransformer([
    ('num', num_preprocessor, num_cols),
    ('cat', cat_preprocessor, cat_cols)
])
```

LLM VERSION

The preprocessing must be designed separately for numerical and categorical features.

For numerical features, I would use median imputation followed by standard scaling. Median imputation handles missing values without being overly influenced by outliers. Standard scaling transforms each numerical feature so that it has approximately zero mean and unit variance. This is especially important for models such as logistic regression and support vector machines, because these models are sensitive to feature scale.

For categorical features, I would use most-frequent imputation followed by one-hot encoding. Most-frequent imputation replaces missing categorical values with the most common category in the training data. One-hot encoding then converts each category into binary indicator columns. This allows models to use categorical information without imposing an artificial numerical ordering.

These preprocessing steps should be placed inside a `ColumnTransformer`, which allows numerical and categorical columns to be processed differently. The `ColumnTransformer` should then be placed inside a `Pipeline` together with the machine learning model. This is one of the most important parts of the workflow because it prevents data leakage.

Data leakage would occur if imputation values, scaling parameters, or encoding decisions were learned from the full dataset before cross-validation. By putting preprocessing inside the pipeline, each cross-validation fold learns preprocessing only from the training portion of that fold.

COMMENTS

Me and ChatGPT had the same ideas for preprocessing and I think that is also because of the anonymity of the data, meaning that we cannot put too much effort

into cleaning the dataset. We both used pipelines which is the standard approach for preprocessing.

PRINCIPAL COMPONENT ANALYSIS

Since we have many highly correlated dimensions, I try to reduce dimensionality of my dataset using `PCA` (Principal Component Analysis).

```
In [13]: pca_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler()),
    ('pca', PCA(n_components=0.95, random_state=random_state))
])

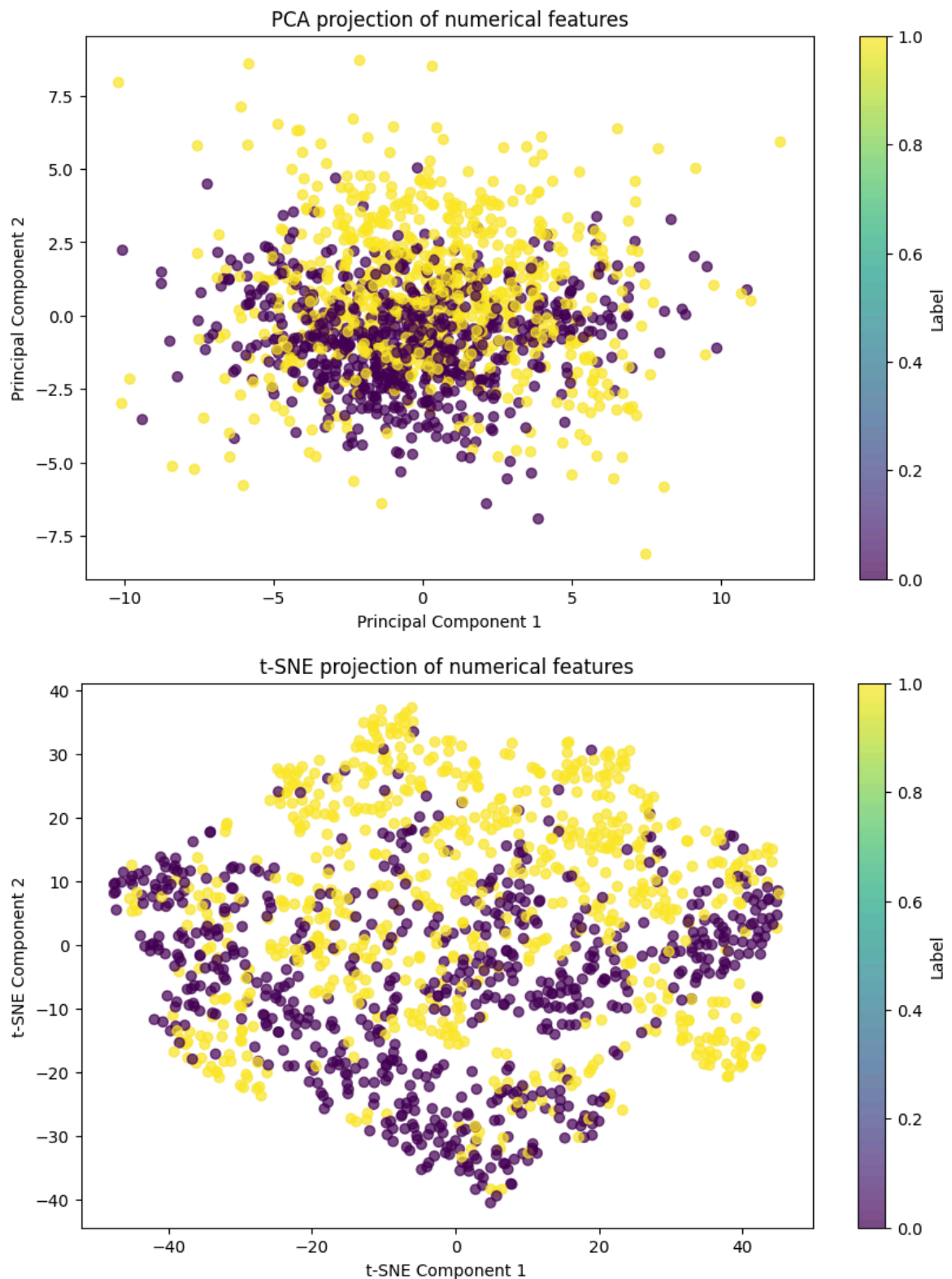
x_num_pca = pca_pipeline.fit_transform(x_train[num_cols])

pca_pipeline.named_steps['pca'].n_components_

plt.figure(figsize=(10, 6))
plt.scatter(x_num_pca[:, 0], x_num_pca[:, 1], c=y_train, cmap='viridis',
plt.colorbar(label='Label')
plt.title('PCA projection of numerical features')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.show()

tsne_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler()),
    ('tsne', TSNE(n_components=2, random_state=random_state))
])

x_num_tsne = tsne_pipeline.fit_transform(x_train[num_cols])
plt.figure(figsize=(10, 6))
plt.scatter(x_num_tsne[:, 0], x_num_tsne[:, 1], c=y_train, cmap='viridis')
plt.colorbar(label='Label')
plt.title('t-SNE projection of numerical features')
plt.xlabel('t-SNE Component 1')
plt.ylabel('t-SNE Component 2')
plt.show()
```



The **PCA** plot shows the first two principal components of the numeric features, colored by the label. The two classes overlap substantially, so a two-component linear projection does not reveal clear class separation. The **t-SNE** plot is a nonlinear two-dimensional visualization of the numeric feature space. Unlike **PCA**, **t-SNE** focuses on preserving local neighborhoods. The plot shows some local regions where one label is more common, but the two classes are still mixed overall. Therefore, these plots are useful for exploratory visualization, but final model choice should be based on cross-validation rather than visual separation. This provides evidence supporting the implementation of non-linear models that may be able to

capture non-linear relations between features, such as `Catboost` or `RandomForest`.

LLM VERSION

//

COMMENTS

ChatGPT didn't use principal component analysis, I think this is mainly because he didn't inspect further the correlation between features. A posteriori I can say that this can be considered a light mistake as we have will see that PCA yields promising results.

MODEL SELECTION

I evaluate the following mandatory models:

- `Logistic Regression`
- `Support Vector Classifier with a linear kernel`
- `Linear Support Vector Classifier`

Logistic Regression is a linear classifier that models the probability of belonging to class 1:

$$P(y = 1 \mid x) = \frac{1}{1 + e^{-(\beta_0 + \beta^T x)}}$$

The support vector classifiers also use a linear decision function:

$$f(x) = w^T x + b$$

where the predicted class depends on the sign of $f(x)$. The SVM objective tries to find a separating hyperplane with a large margin between the two classes. For both Logistic Regression and SVM models, the parameter C controls the strength of regularization: smaller values of C imply stronger regularization. Logistic Regression optimizes log loss, while the SVM models optimize hinge-based margin losses. In particular, `LinearSVC` uses squared hinge loss by default. I acknowledge that all three models use different loss functions, Logistic Regression optimizes log loss, while the SVM models optimize hinge-based margin losses. I do not change the loss functions as they are not relevant tuning dimensions (for two out of the three models the loss models is fixed). For the model selection I will rely on `balanced_accuracy` as a metric because it was the metric of the classification extra point challenge. The whole section of model selection is being done on a second split of the training data, to keep an "unseen" evaluation set to get a more realistic evaluation metric after fine tuning the parameters.

```
In [14]: models = {
    'LogisticRegression' : Pipeline([
        ('preprocessor', preprocessor),
        ('model', LogisticRegression(random_state=random_state, max_iter=
    ]),
    'SVC' : Pipeline([
        ('preprocessor', preprocessor),
        ('model', SVC(kernel='linear', random_state=random_state))
    ]),
    'LinearSVC' : Pipeline([
        ('preprocessor', preprocessor),
        ('model', LinearSVC(random_state=random_state, max_iter=20000, du
    ])
}
x_tr, x_val, y_tr, y_val = train_test_split(x_train, y_train, test_size=0
eval_set = (x_val, y_val) # I will keep this set aside for final evaluati
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=random_state)
y_tr.value_counts(normalize=True), y_val.value_counts(normalize=True) # I
```

```
Out[14]: (label
1      0.5125
0      0.4875
Name: proportion, dtype: float64,
label
1      0.510714
0      0.489286
Name: proportion, dtype: float64)
```

```
In [15]: print("\nCross-validation results:")
cross_val_scores = {}
for name, model in models.items():
    scores = cross_val_score(
        model,
        x_tr,
        y_tr,
        cv=cv,
        scoring='balanced_accuracy'
    )
    cross_val_scores[name] = scores.mean()
    print(f'{name} - Balanced Accuracy: {scores.mean():.4f}')
```

Cross-validation results:
 LogisticRegression - Balanced Accuracy: 0.7873
 SVC - Balanced Accuracy: 0.7872
 LinearSVC - Balanced Accuracy: 0.7881

The obtained results signify that the Linear SVM models perform similarly, as does the Logistic Regression. My hypothesis is that the model struggles to identify non-linear relationships between the features, as highlighted by PCA, so I will try and add Polynomial Features to Logistic Regression

POLYNOMIAL FEATURES

We now check whether my hypothesis is correct by running a Logistic Regression with the implementation of Polynomial Features.

```
In [16]: lognum_preprocessor = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler()),
    ('poly', PolynomialFeatures(degree=2, interaction_only=True, include_
    ('secondscaler', StandardScaler())
])

logistic_preprocessor = ColumnTransformer([
    ('num', lognum_preprocessor, num_cols),
    ('cat', cat_preprocessor, cat_cols)
])

logregpol_model = Pipeline([
    ('preprocessor', logistic_preprocessor),
    ('model', LogisticRegression(random_state=random_state, max_iter=2000
])

pol_score = cross_val_score(logregpol_model, x_tr, y_tr, cv=cv, scoring='
print(f'Logistic Regression with polynomial features - Balanced Accuracy:
```

Logistic Regression with polynomial features - Balanced Accuracy: 0.8287

As predicted, the balanced accuracy score is higher.

EXTRA MODELS

```
In [17]: try:
    import catboost
    print("catboost already installed")
except ImportError:
    %pip install catboost

from catboost import CatBoostClassifier
```

catboost already installed

As mentioned above, I will include two Tree-based models which should have better performance with a dataset of this structure.

```
In [18]: rnd_forest = Pipeline([
    ('preprocessor', preprocessor),
    ('model', RandomForestClassifier(random_state=random_state))
])

catboost_model = Pipeline([
    ('preprocessor', preprocessor),
    ('model', CatBoostClassifier(random_state=random_state, verbose=10, i
])

rnd_forest_scores = cross_val_score(
    rnd_forest,
    x_tr,
    y_tr,
    cv=cv,
    scoring='balanced_accuracy'
)
```

```
catboost_scores = cross_val_score(  
    catboost_model,  
    x_tr,  
    y_tr,  
    cv=cv,  
    scoring='balanced_accuracy'  
)  
  
print(f'Random Forest - Balanced Accuracy: {rnd_forest_scores.mean():.4f}')  
print(f'CatBoost - Balanced Accuracy: {catboost_scores.mean():.4f}')
```

Learning rate set to 0.081203

0:	learn: 0.6618408	total: 58.5ms	remaining: 5.79s
10:	learn: 0.4846949	total: 72.7ms	remaining: 588ms
20:	learn: 0.4093022	total: 84.7ms	remaining: 319ms
30:	learn: 0.3556703	total: 96.2ms	remaining: 214ms
40:	learn: 0.3243553	total: 111ms	remaining: 160ms
50:	learn: 0.2912581	total: 128ms	remaining: 123ms
60:	learn: 0.2606123	total: 145ms	remaining: 92.9ms
70:	learn: 0.2387649	total: 158ms	remaining: 64.6ms
80:	learn: 0.2151767	total: 172ms	remaining: 40.4ms
90:	learn: 0.1955036	total: 184ms	remaining: 18.2ms
99:	learn: 0.1779349	total: 198ms	remaining: 0us

Learning rate set to 0.081203

0:	learn: 0.6662851	total: 2.5ms	remaining: 248ms
10:	learn: 0.5102119	total: 20.8ms	remaining: 168ms
20:	learn: 0.4349079	total: 36.1ms	remaining: 136ms
30:	learn: 0.3863946	total: 49.2ms	remaining: 110ms
40:	learn: 0.3483418	total: 60.9ms	remaining: 87.7ms
50:	learn: 0.3158348	total: 74.5ms	remaining: 71.6ms
60:	learn: 0.2868008	total: 87.3ms	remaining: 55.8ms
70:	learn: 0.2643617	total: 99.8ms	remaining: 40.8ms
80:	learn: 0.2393113	total: 113ms	remaining: 26.4ms
90:	learn: 0.2153178	total: 125ms	remaining: 12.4ms
99:	learn: 0.1968614	total: 139ms	remaining: 0us

Learning rate set to 0.081203

0:	learn: 0.6653887	total: 2.59ms	remaining: 256ms
10:	learn: 0.5031920	total: 17.7ms	remaining: 143ms
20:	learn: 0.4156970	total: 33.2ms	remaining: 125ms
30:	learn: 0.3680601	total: 50.6ms	remaining: 113ms
40:	learn: 0.3280353	total: 66.5ms	remaining: 95.7ms
50:	learn: 0.2963634	total: 81.4ms	remaining: 78.2ms
60:	learn: 0.2650157	total: 96.2ms	remaining: 61.5ms
70:	learn: 0.2469070	total: 111ms	remaining: 45.2ms
80:	learn: 0.2245823	total: 124ms	remaining: 29.2ms
90:	learn: 0.2030188	total: 138ms	remaining: 13.6ms
99:	learn: 0.1840405	total: 150ms	remaining: 0us

Learning rate set to 0.081203

0:	learn: 0.6648068	total: 1.27ms	remaining: 126ms
10:	learn: 0.4940278	total: 14.7ms	remaining: 119ms
20:	learn: 0.4146283	total: 28.7ms	remaining: 108ms
30:	learn: 0.3657112	total: 43.1ms	remaining: 95.9ms
40:	learn: 0.3251645	total: 58.8ms	remaining: 84.7ms
50:	learn: 0.3000283	total: 72.7ms	remaining: 69.8ms
60:	learn: 0.2715641	total: 86.7ms	remaining: 55.4ms
70:	learn: 0.2430177	total: 98.7ms	remaining: 40.3ms
80:	learn: 0.2169727	total: 111ms	remaining: 26.1ms
90:	learn: 0.1942797	total: 124ms	remaining: 12.2ms
99:	learn: 0.1794170	total: 135ms	remaining: 0us

Learning rate set to 0.081203

0:	learn: 0.6642633	total: 1.19ms	remaining: 118ms
10:	learn: 0.4866460	total: 16.1ms	remaining: 130ms
20:	learn: 0.4088314	total: 30.4ms	remaining: 114ms
30:	learn: 0.3585918	total: 44.2ms	remaining: 98.4ms
40:	learn: 0.3251090	total: 57.2ms	remaining: 82.3ms
50:	learn: 0.2921677	total: 68.6ms	remaining: 65.9ms
60:	learn: 0.2676718	total: 81ms	remaining: 51.8ms
70:	learn: 0.2454114	total: 96.5ms	remaining: 39.4ms
80:	learn: 0.2201491	total: 107ms	remaining: 25.2ms
90:	learn: 0.1972014	total: 119ms	remaining: 11.8ms
99:	learn: 0.1778141	total: 130ms	remaining: 0us

Random Forest – Balanced Accuracy: 0.8142

CatBoost – Balanced Accuracy: 0.8221

We can find the final results in the table below: we can see how the models able to better capture non linear features perform better than the linear ones.

```
In [19]: model_results = {
    'Logistic Regression': cross_val_scores['LogisticRegression'],
    'SVC': cross_val_scores['SVC'],
    'LinearSVC': cross_val_scores['LinearSVC'],
    'Polynomial Logistic Regression': pol_score,
    'Random Forest': rnd_forest_scores.mean(),
    'CatBoost': catboost_scores.mean()
}

model_results_df = pd.DataFrame.from_dict(model_results, orient='index',
model_results_df.sort_values(by='Balanced Accuracy', ascending=False, inplace=True)
model_results_df.head()
```

```
Out [19]:
```

	Balanced Accuracy
Polynomial Logistic Regression	0.828679
CatBoost	0.822074
Random Forest	0.814200
LinearSVC	0.788125
Logistic Regression	0.787319

LLM VERSION

After preprocessing was defined, I compared several classification models. The goal of this stage was to identify which type of model was most suitable for the dataset. I started with a dummy classifier as a baseline. This model does not learn meaningful patterns and usually predicts the majority class, so it provides a minimum reference score. Any useful machine learning model should clearly outperform this baseline.

I then tested a set of more serious models. Logistic regression was included as a simple and interpretable linear model. It is useful as a strong baseline because it often performs well on tabular data after proper preprocessing. However, because it is linear, it may not fully capture complex nonlinear relationships between the features and the target.

I also tested a support vector classifier with an RBF kernel. This model can learn nonlinear decision boundaries and is often effective when the dataset has a moderate number of observations and features. Since support vector machines depend heavily on feature scale, the scaling step in preprocessing was especially important for this model.

To compare the models fairly, I used stratified k-fold cross-validation. Stratification ensured that each fold preserved approximately the same class distribution as the original dataset. I evaluated the models using several metrics, with ROC AUC as the

main selection criterion. ROC AUC was chosen because it measures how well the model separates the two classes across different classification thresholds. I also considered accuracy, F1-score, and average precision to make sure the selected model performed well from different perspectives.

Based on the comparison, the RBF support vector classifier appeared to be the strongest model, so it was chosen as the main candidate for hyperparameter tuning.

COMMENTS

The LLM ignored the request of only using the linear kernel and opted for the RBF SVC. As I mentioned after PCA, a non linear model may be better for this dataset because there seem to be non linear relationships between features.

HYPERPARAMETER TUNING

We continue with the finetuning of the three models although the ideal models would be Polynomial Logistic Regression or Catboost/RandomForest but they are not in the scope of this assignment. Now we run 100 iterations of RandomizedSearchCV to find the best parameters. We use only random search and do not implement grid search because given the same evaluation budget, random search often outperforms grid search for hyperparameter optimization, especially in high-dimensional spaces where only a few hyperparameters strongly affect performance. (I used this paper as reference "Random Search for Hyper-Parameter Optimization" by James Bergstra, Yoshua Bengio)

```
In [20]: search_results = {}

for name, model in models.items():
    param_dist = {}

    if name == 'LogisticRegression':
        param_dist = {
            'model__C': loguniform(1e-2, 100), # i do not tune penalty be
            'model__class_weight': ['balanced', None]
        }

    elif name == 'SVC':
        param_dist = {
            'model__C': loguniform(1e-2, 100),
            'model__kernel': ['linear'],
            'model__class_weight': ['balanced', None]
        }

    elif name == 'LinearSVC':
        param_dist = {
            'model__C': loguniform(1e-2, 100),
            'model__penalty': ['l2'],
            'model__dual': [False],
            'model__class_weight': ['balanced', None]
        }
```

```

random_search = RandomizedSearchCV(
    model,
    param_distributions=param_dist,
    n_iter=30,
    cv=cv,
    scoring='balanced_accuracy',
    random_state=random_state,
    error_score='raise'
)

random_search.fit(x_tr, y_tr)
search_results[name] = random_search
print(
    f'{name} - Best Balanced Accuracy: {random_search.best_score_:.4f}'
    f'with params: {random_search.best_params_}'
)

```

LogisticRegression - Best Balanced Accuracy: 0.7902 with params: {'model__C': np.float64(15.847892724427277), 'model__class_weight': 'balanced'}

SVC - Best Balanced Accuracy: 0.7889 with params: {'model__C': np.float64(0.6618463633643242), 'model__class_weight': None, 'model__kernel': 'linear'}

LinearSVC - Best Balanced Accuracy: 0.7908 with params: {'model__C': np.float64(3.8516095034638993), 'model__class_weight': None, 'model__dual': False, 'model__penalty': 'l2'}

LLM VERSION

Once the best model family was selected, I tuned its hyperparameters to improve performance. For the RBF support vector classifier, the two most important hyperparameters are C and gamma.

The parameter C controls the amount of regularization. A smaller value of C creates a smoother and more regularized decision boundary, while a larger value allows the model to fit the training data more closely. If C is too large, the model may overfit. If it is too small, the model may underfit.

The parameter gamma controls how much influence each training observation has. A small value of gamma produces a smoother decision boundary, while a larger value creates a more complex and local decision boundary. As with C, choosing an inappropriate value can lead to either underfitting or overfitting.

I used cross-validated grid search to test different combinations of C and gamma. The full preprocessing and modeling pipeline was included inside the grid search, so each parameter combination was evaluated correctly without leaking information between training and validation folds.

The tuning process selected the combination of hyperparameters that gave the best cross-validated ROC AUC. After tuning, I evaluated the selected model on a held-out validation set using ROC AUC, average precision, accuracy, precision, recall, F1-score, and a confusion matrix. I also used ROC and precision-recall curves to visualize the model's behavior across different thresholds.

This tuning stage helped ensure that the final model was not chosen only because of its default settings, but because its most important parameters had been systematically optimized.

COMMENTS

The LLM fine-tunes a model I didn't consider so I cannot make an appropriate comparison. The only relevant thing I can say is that it uses as a metric ROC AUC. I initially wanted to use this metric as well, as it's industry standard, but then opted for balanced accuracy to be consistent with the extra point challenge. Moreover, he did not consider PCA as he did in the preprocessing section.

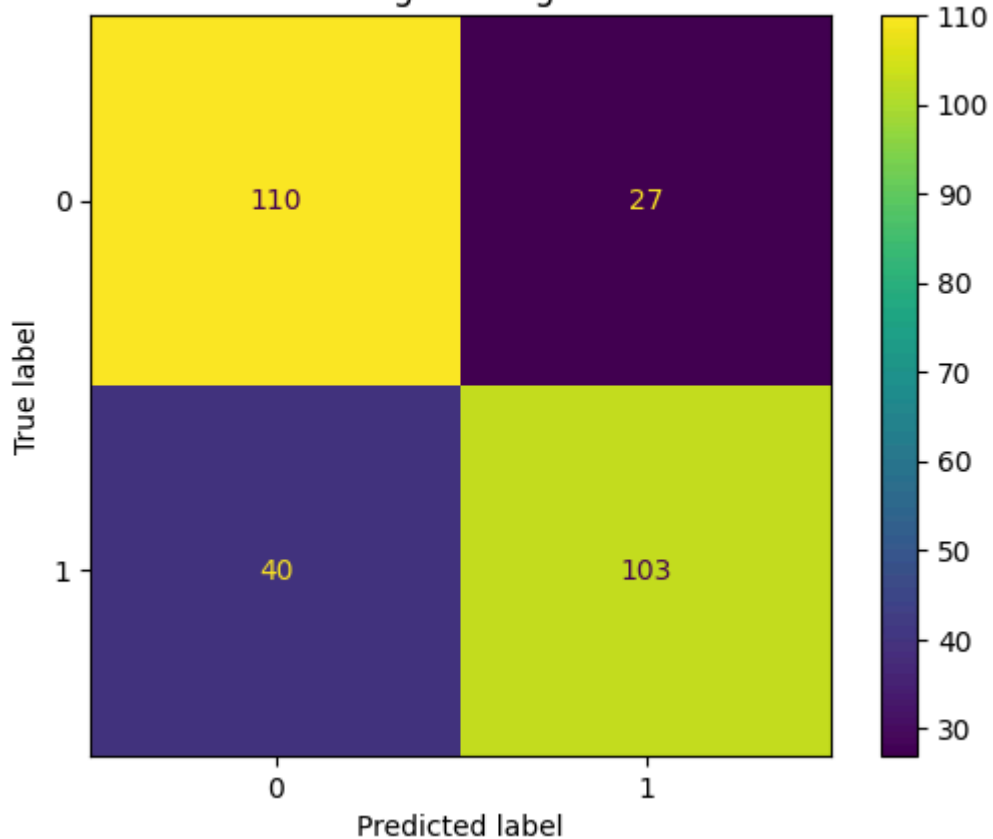
FITTING THE MODEL

We select Logistic Regression as the final model. Although none of the models scores clearly better than the others, Logistic Regression is a good choice because it provides probability estimates, which are useful for evaluating the classifier with ROC AUC, which we will do later (as I've been told in my work-related experience that AUC is the industry standard). Before fitting the final model on the full training data, we compare its performance on the evaluation set created earlier. We also check the results of the two additional models on the same evaluation set.

```
In [21]: bestlog_reg_model = search_results['LogisticRegression'].best_estimator_  
bestlog_reg_model = Pipeline([  
    ('preprocessor', preprocessor),  
    ('model', LogisticRegression(random_state=random_state, max_iter=2000  
    ]))  
  
bestlog_reg_model.fit(x_tr, y_tr)  
y_pred = bestlog_reg_model.predict(x_val)  
  
print("Logistic Regression evaluation on validation set:")  
print(f"Balanced Accuracy: {balanced_accuracy_score(y_val, y_pred):.4f}")  
  
conf_matrix = ConfusionMatrixDisplay.from_estimator(bestlog_reg_model, x_  
plt.title("Confusion Matrix of the Logistic Regression on Validation Set")  
plt.show()
```

Logistic Regression evaluation on validation set:
Balanced Accuracy: 0.7616

Confusion Matrix of the Logistic Regression on Validation Set



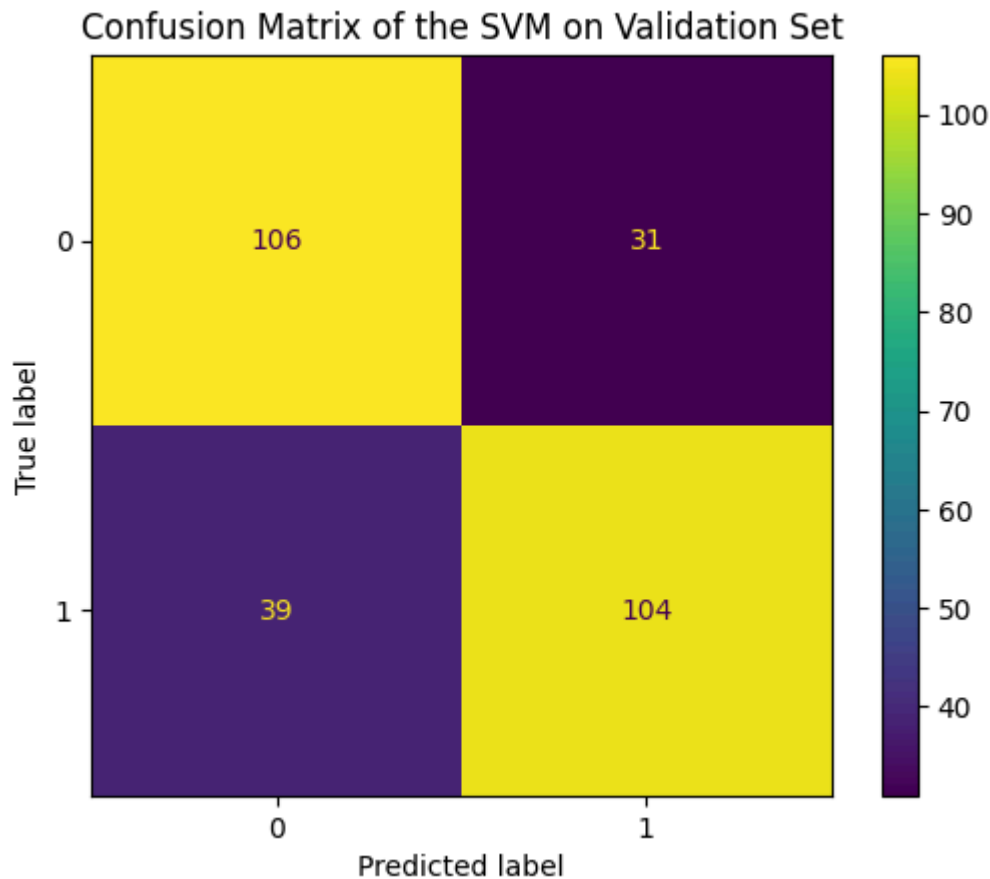
```
In [22]: best_svm_model = search_results['SVC'].best_estimator_
best_linearsvc_model = search_results['LinearSVC'].best_estimator_
best_svm_model = Pipeline([
    ('preprocessor', preprocessor),
    ('model', SVC(kernel='linear', random_state=random_state, C=best_svm_
)])

best_linearsvc_model = Pipeline([
    ('preprocessor', preprocessor),
    ('model', LinearSVC(random_state=random_state, max_iter=20000, C=best
)])

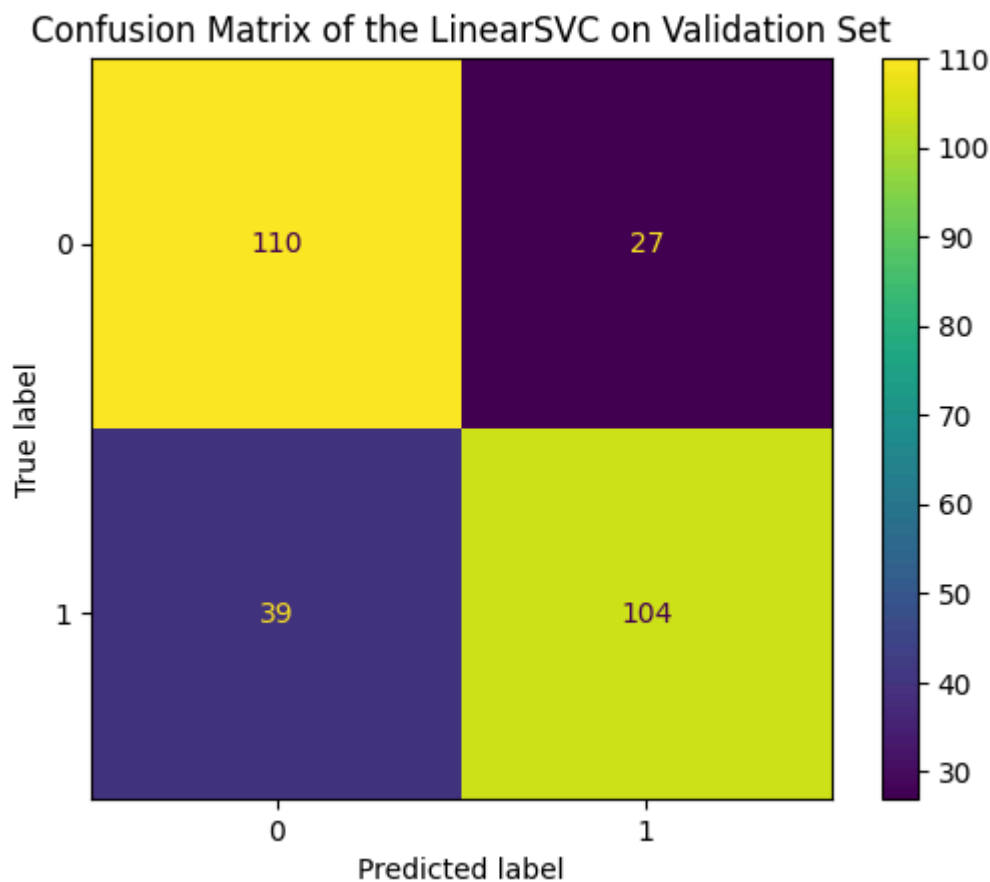
best_svm_model.fit(x_tr, y_tr)
y_svm_pred = best_svm_model.predict(x_val)
print("SVM evaluation on validation set:")
print(f"balanced accuracy: {balanced_accuracy_score(y_val, y_svm_pred):.4}
ConfusionMatrixDisplay.from_estimator(best_svm_model, x_val, y_val, displ
plt.title("Confusion Matrix of the SVM on Validation Set")
plt.show()

best_linearsvc_model.fit(x_tr, y_tr)
y_linearsvc_pred = best_linearsvc_model.predict(x_val)
print("LinearSVC evaluation on validation set:")
print(f"balanced accuracy: {balanced_accuracy_score(y_val, y_linearsvc_pr
ConfusionMatrixDisplay.from_estimator(best_linearsvc_model, x_val, y_val,
plt.title("Confusion Matrix of the LinearSVC on Validation Set")
plt.show()
```

SVM evaluation on validation set:
balanced accuracy: 0.7505



LinearSVC evaluation on validation set:
balanced accuracy: 0.7651



All models perform similarly on the unseen data, good news. With LinearSVC performing slightly better than the others. Nonetheless I now include the ROC_AUC

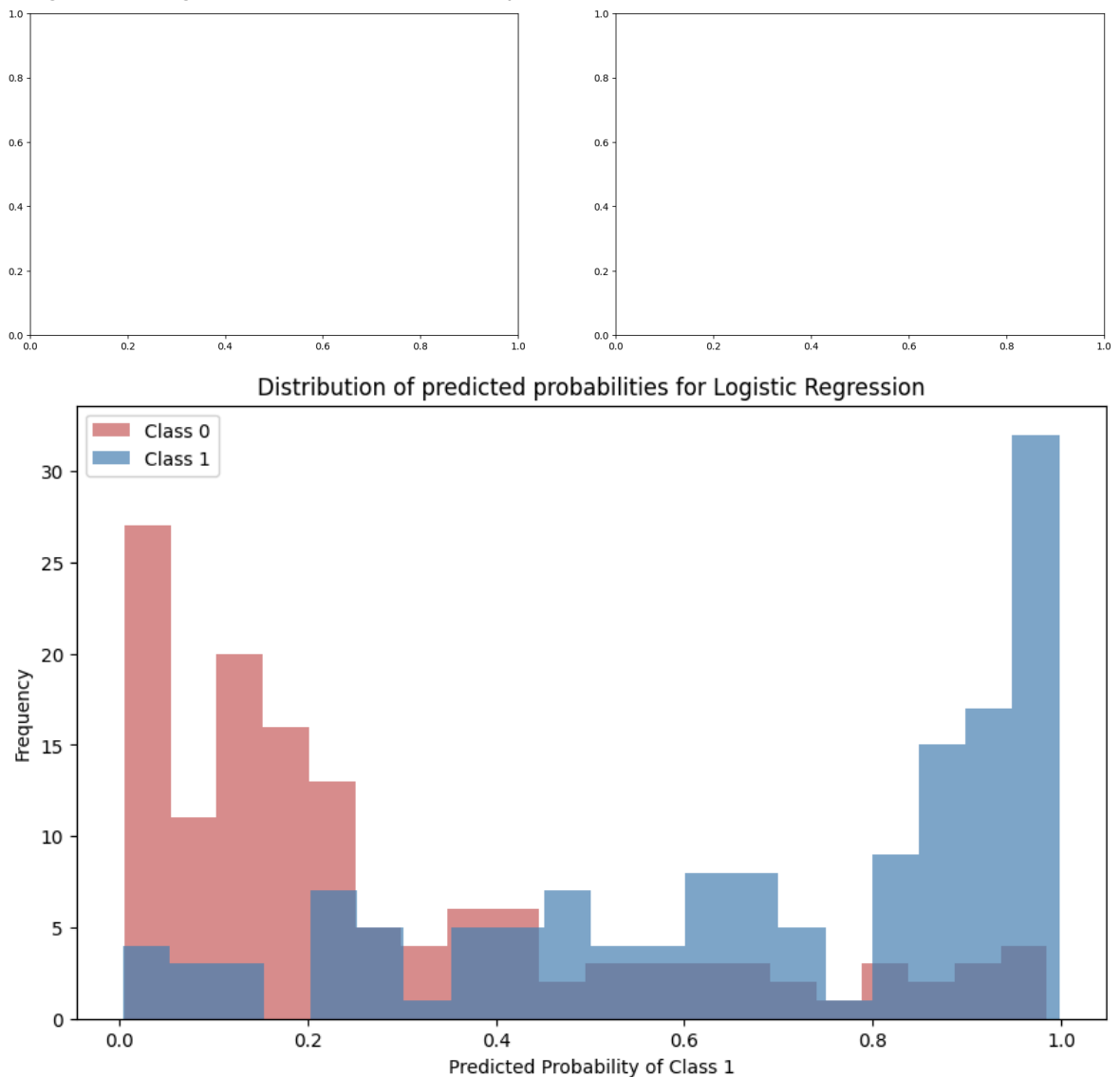
scores for Logistic Regression to cover all my bases.

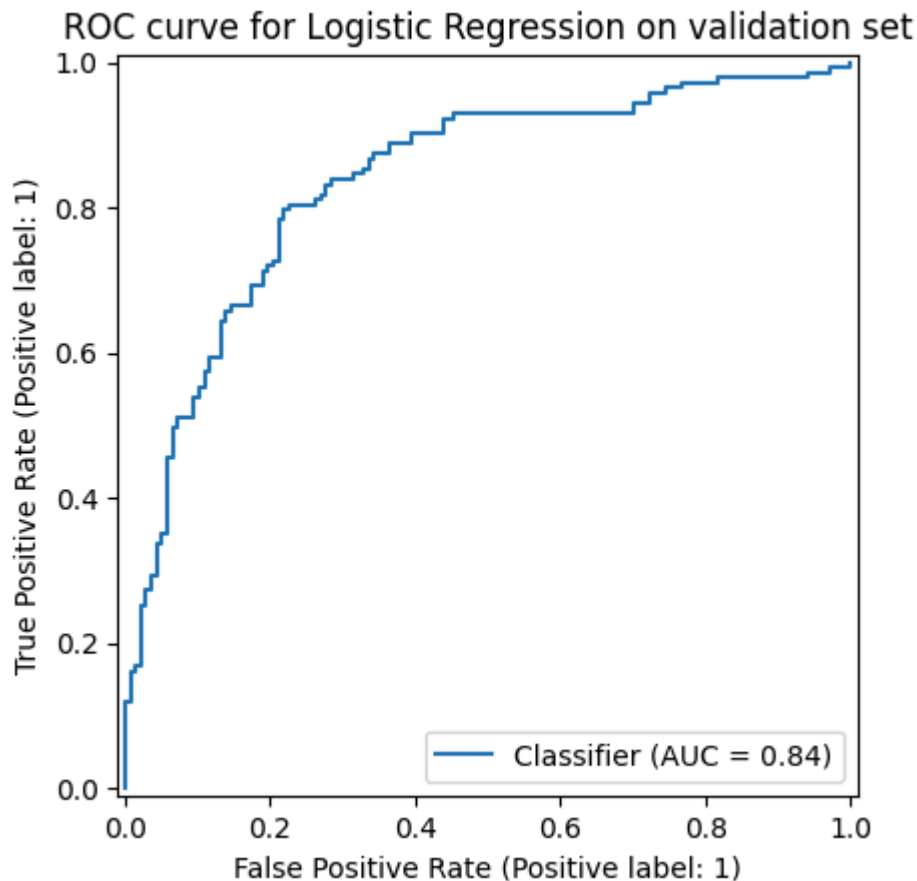
```
In [23]: y_log_proba = bestlog_reg_model.predict_proba(x_val)[: , 1]
print(f"Logistic Regression ROC AUC with probabilities: {roc_auc_score(y_val, y_log_proba)}")

fig, axes = plt.subplots(1, 2, figsize=(20, 6))
plt.figure(figsize=(10, 6))
plt.hist(y_log_proba[y_val == 0], bins=20, alpha=0.7, label='Class 0', color='red')
plt.hist(y_log_proba[y_val == 1], bins=20, alpha=0.7, label='Class 1', color='blue')
plt.title('Distribution of predicted probabilities for Logistic Regression')
plt.xlabel('Predicted Probability of Class 1')
plt.ylabel('Frequency')
plt.legend()
plt.show()

RocCurveDisplay.from_predictions(y_val, y_log_proba)
plt.title("ROC curve for Logistic Regression on validation set")
plt.show()
```

Logistic Regression ROC AUC with probabilities: 0.8368





PCA SCORES

We now try to perform the analysis on the reduced dataset, which contains 7 numerical features (the ones necessary to explain 95% of variance) and two categorical features.

```
In [24]: pca_cols = [f"PC{i+1}" for i in range(x_num_pca.shape[1])]

x_pca_df = pd.DataFrame(
    x_num_pca,
    columns=pca_cols,
    index=x_train.index
)

x_pca_cat = pd.concat([x_pca_df, x_train[cat_cols]], axis=1) # we concatenate

num_pca_preprocessor = Pipeline([
    ('num_preprocessor', num_preprocessor),
    ('pca', PCA(n_components=0.95, random_state=random_state))
])

pca_preprocessor = ColumnTransformer([
    ('num', num_pca_preprocessor, num_cols),
    ('cat', cat_preprocessor, cat_cols)
])

x_pca_cat.head()
```


Out [24]:

	PC1	PC2	PC3	PC4	PC5	PC6	PC7	PC8
0	5.937349	-3.081484	3.046518	-0.711188	-0.630098	1.020210	-0.205387	
1	-5.703926	-2.173680	-1.089319	1.946461	-0.044532	0.584485	-0.505383	
2	-1.095615	0.681505	0.630946	-0.537679	-0.437568	-0.017782	0.138875	
3	-3.712646	-1.038568	1.254724	2.640167	-0.446685	-0.171908	-0.705434	
4	-2.829024	-2.497424	-0.286902	0.703684	2.591134	-0.327190	-0.015392	

In [25]:

```

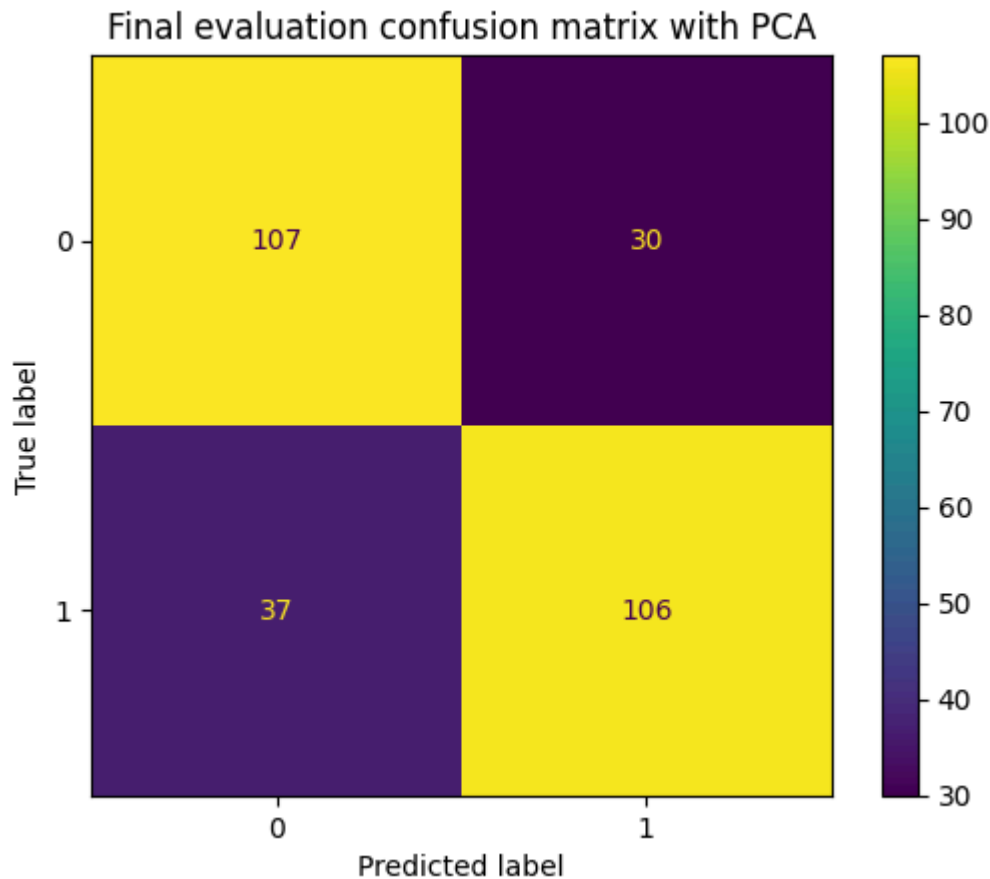
pca_model = Pipeline([
    ('preprocessor', pca_preprocessor),
    ('model', LogisticRegression(
        random_state=random_state,
        max_iter=20000,
        C=15.85,
        class_weight='balanced'
    ))
])

pca_model.fit(x_tr, y_tr)
y_pca_pred = pca_model.predict(x_val)

print("Final evaluation results with PCA:")
print("Balanced accuracy:", balanced_accuracy_score(y_val, y_pca_pred))
print()
ConfusionMatrixDisplay.from_predictions(y_val, y_pca_pred)
plt.title("Final evaluation confusion matrix with PCA")
plt.show()

```

Final evaluation results with PCA:
Balanced accuracy: 0.7611403195344801



The results of the PCA analysis yield only a 4% decrease in balanced accuracy with respect to the LinearSVC model we trained before, so we can say that model reduction is useful for this dataset.

LLM VERSION

After completing model selection and hyperparameter tuning, I trained the final model on the full labeled training dataset. This was done only after the validation and tuning process was complete. The reason for refitting on the full training data is that, once the best modeling strategy has been chosen, the model should use all available labeled examples to learn the final decision boundary.

The final fitted pipeline included all preprocessing steps and the tuned classifier. This means that the same operations used during cross-validation were also applied during final training and prediction. Numerical missing values were imputed, numerical features were scaled, categorical missing values were imputed, categorical features were one-hot encoded, and then the tuned classifier was applied.

Finally, the fitted model was used to generate predictions for the hidden test set. Since the hidden test set did not contain labels, it was not used for evaluation or model selection. It was only used at the final stage, after the model had already been chosen and trained. The output consisted of the test-set IDs together with the predicted class labels.

This final fitting strategy ensures that the hidden test predictions are produced by a model that was selected using a proper validation procedure, trained on all available

labeled data, and applied consistently through a leakage-free preprocessing pipeline.

COMMENTS

ChatGPT used a standard baseline, as I did. He trained the model on the whole training dataset. The one thing he didn't do is keeping an evaluation set to understand a more realist value for the accuracy scores.

SHAP VISUALIZATION

I now include a very brief section of shap visualization, as I'm currently working with it in an uncorrelated project and I find it interesting and explanatory for binary prediction. I will use as a model RandomForest as, although it performs slightly worse than Logistic Regression, it's easier to explain because features and columns remain similar (apart from the encoding for categorical features), namely because the expanded feature space contains many interaction terms, making the explanations harder to interpret.

```
In [26]: try:
import shap
print("shap already installed")
except ImportError:
!pip install shap
```

shap already installed

```
In [27]: rnd_forest.fit(x_tr, y_tr)

X_shap_transformed = rnd_forest.named_steps['preprocessor'].transform(x_val)
feature_names = rnd_forest.named_steps['preprocessor'].get_feature_names_out()

X_shap = pd.DataFrame(
    X_shap_transformed,
    columns=feature_names,
    index=x_val.index
)

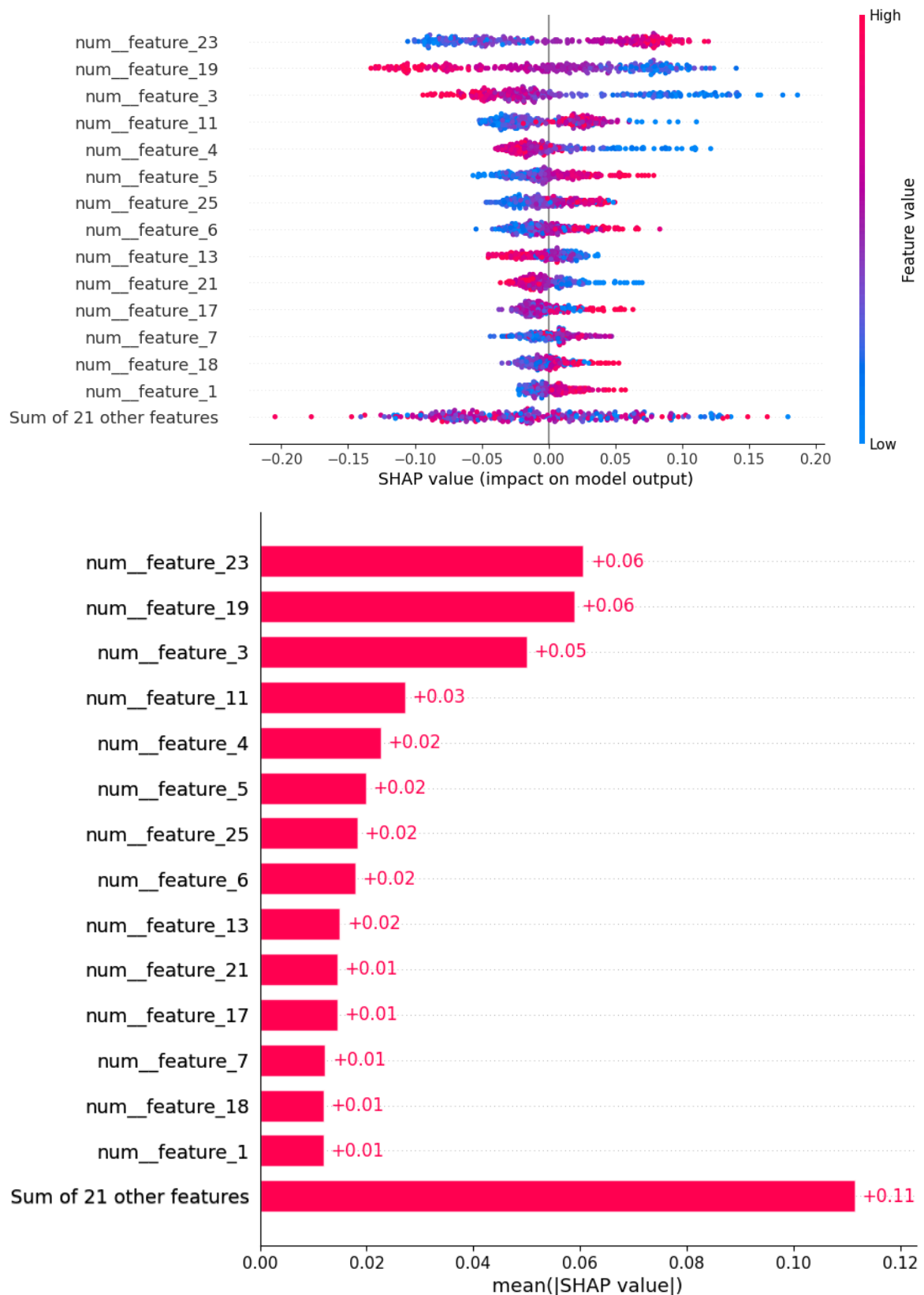
rf_model = rnd_forest.named_steps['model']

explainer = shap.TreeExplainer(rf_model)

shap_values = explainer(X_shap)
shap_values_class1 = shap_values[:, :, 1]

shap.plots.beeswarm(shap_values_class1, max_display=15, plot_size=(10, 6))

shap.plots.bar(shap_values_class1, max_display=15)
```



The SHAP bar plot shows which variables have the largest average impact on the model prediction. The most relevant features are num__feature_23, num__feature_19 and num__feature_3, meaning that these are the variables the model relies on the most when separating the two classes. The beeswarm plot gives more detail on the direction of this effect. Positive SHAP values push the prediction towards label 1, while negative SHAP values push it towards label 0. The colour scale shows the feature value: blue means low values and pink means high values. From the beeswarm plot, num__feature_3 is particularly interesting: high values are mostly

associated with negative SHAP values, so they push the model towards label 0, while low values are mostly associated with positive SHAP values, pushing the model towards label 1. This suggests that `num__feature_3` is a useful indicator for the classification task. Moreover, the shap analysis is consistent with the mutual information one, as in both features 3, 19 and 23 were the most informative ones. In a real-world setting, if labels 0 and 1 represented loan rejection and loan approval, SHAP would help identify which features push the model towards one outcome or the other. For example, if `num__feature_3` represented age, the model would be associating lower values of age with a higher probability of approval and higher values with a higher probability of rejection.

LLM VERSION

//

COMMENTS

Did not do this part.

PREDICTIONS

Here are the final predictions, using the LinearSVC model:

```
In [28]: x_test = pd.read_csv('mldata_0003314701.TEST_FEATURES.csv')
bestlog_reg_model.fit(x_train, y_train)
y_test_pred = bestlog_reg_model.predict(x_test)

final_predictions = pd.DataFrame({
    'id': x_test['id'],
    'label': y_test_pred
})
final_predictions.to_csv('final_predictions.csv', index=False, header = F
```

After having completed my notebook, I asked the LLM to compare the two notebooks. Here are the main differences he spotted:

1. *No discussion of selection bias.* The LLM proposes using the same CV for both hyperparameter search and final reporting, with no mention that the resulting "best score" is biased upward by selection-over-noise. *Nested cross-validation* (or at least repeated CV with reported standard deviations) is the literature-standard fix for honest performance estimates on small data, and the LLM does not raise it. With $n = 1400$ and ~ 50 candidate configurations, the reported CV score for the winning model is materially optimistic relative to true generalization.

2. *No literature-aware model recommendation.* The LLM proposes Logistic Regression and SVM as the primary candidates and adds tree ensembles as optional extras. The published benchmarks point the other way: Grinsztajn et al. (NeurIPS 2022) and the 2024 comprehensive tabular benchmark show that *gradient-boosted decision trees are state-of-the-art* on tabular data of this size and structure, while

linear models are typically dominated by 3–5 points of balanced accuracy. My own results agree — RandomForest reached ~ 0.82 against ~ 0.78 for the linear models. A literature-informed answer would have recommended CatBoost / XGBoost / LightGBM as the default choice and treated the linear models as an interpretable baseline. The mandatory task locks this project to linear models, but the LLM's framing reads as if the linear choice were a substantive recommendation rather than an assignment constraint.

3. *No critique of common defaults.* The LLM suggests `class_weight='balanced'` without checking the class distribution (mine is 51 / 49, essentially balanced — the reweighting is a no-op at best). It also suggests reporting ROC AUC without flagging that on hard-label predictions AUC collapses to balanced accuracy and is misleading. Both are common pitfalls and the LLM does not flag them.

FINAL REMARKS AND LIMITATIONS

In conclusion, the analysis of this dataset is definitely affected by the anonymity of the features which definitely limited feature engineering and data preprocessing. This means that, even if the model can recognise useful statistical patterns, we cannot fully explain why those patterns exist or whether they make sense from a practical point of view. This also limits the interpretation of the SHAP results. The plots help us understand which features influence the model the most, but they cannot tell us whether these influences are reasonable, fair, or meaningful outside the dataset.

The LLM that was used for the audit is Chat GPT 5.5 (Edu licence)